

SEMESTER TRAINING (BTCS 801-18)

REPORT

on

Buildr: AI-Powered Website Builder Using Next.js and OpenAI

Submitted in partial fulfilment of requirements for the award of the degree

Bachelor of Technology

In

Information Technology

To

IKG Punjab Technical University, Jalandhar

SUBMITTED BY:

Name: Rajat Khurana

Roll No.: 2237440

Semester: 8th

Batch: 2022-26

Under the Guidance

Mr. Amitabh Sharma and Mr. Kamaljit Singh

Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

Chandigarh Engineering College- CGC

Landran, Mohali, Punjab – 140307

May 2026

DECLARATION

I hereby declare that the Project Report entitled (Buildr: AI-Powered Website Builder Using Next.js and OpenAI) is an authentic record of my own work as requirements of 8th semester academics as Semester Training BTCS 801-18 during the period from January 2026 to May 2026 for the award of degree of B. Tech. Information Technology, Chandigarh Engineering College- CGC Landran, Mohali under the guidance of Mr. Amitabh Sharma and Mr. Kamaljit Singh

NAME: Rajat Khurana

Roll No: 2237440

Date:

Signatures

Examined by:

1.

2.

3.

4.

Head of Department

(Signature and Seal)

Acknowledgement

I take this opportunity to express my sincere gratitude to the Director- Principal **Dr. Rajdeep Singh**, Chandigarh Engineering College-CGC Landran, Mohali for providing this opportunity to carry out the present work.

I am highly grateful to **Dr. Amanpreet Kaur** HOD IT, Chandigarh Engineering College-CGC Landran, Mohali for her invaluable support. I would like to express my gratitude to other faculty members of Information Technology department of CEC-CGC, Landran, Mohali for providing academic input, guidance & Encouragement throughout the training period.

The help rendered by **Mr. Amitabh Sharma and Mr. Kamaljit Singh** Supervisor for training work is greatly acknowledged. Finally, I express my indebtedness to all who have directly or indirectly contributed to the successful completion of my semester training.

About Institute

Institute Overview

Chandigarh Engineering College is a premier educational institution dedicated to providing quality technical education to students across various disciplines. Established in 2001, the institute has consistently strived to foster academic excellence, innovation, and research among its students.

Vision

To become a leading institution of technical education that nurtures talented professionals equipped with knowledge, skills, and values to contribute meaningfully to society and industry.

Mission

- To provide quality education through innovative teaching methodologies
- To encourage research and development among students and faculty
- To prepare students for the challenges of the ever evolving technology landscape
- To foster a culture of creativity, critical thinking, and problem solving

Department of Information Technology

The Department of Information Technology at Chandigarh Engineering College offers a comprehensive B.Tech program designed to equip students with strong theoretical foundations and practical skills in areas including:

- Software Development
- Database Management
- Artificial Intelligence and Machine Learning
- Web Technologies
- Computer Networks

The department is equipped with modern laboratories, experienced faculty, and industry-oriented curriculum that prepares students for successful careers in the technology sector.

Table of Contents

Chapter	Title	Page No.
-	Acknowledgement	i
-	About Institute	ii
-	Table of Contents	iii
-	List of Figures	vi
-	List of Tables	vii
-	Abbreviations and Nomenclature	viii
-	Abstract of the Project	ix
1	Introduction	1
1.1	Brief Overview of Work	2
1.2	Objective	3
1.3	Scope	4
1.4	Project Modules	5
1.5	Project Requirements	7
2	System Analysis	9
2.1	Literature Review	10
2.2	Project Feasibility Study	12
2.4	Project Timeline Chart	14
2.5	Detailed Module Description with all Functionalities	15
3	System Design	18
3.1	Use Case Diagrams	19
3.2	Data Flow Diagrams	21
3.3	Class Diagram	24
3.4	Entity Relationship Diagram	26
3.5	Activity Diagram	28
3.6	Data Dictionary	30
4	Software Tools	33
5	Implementation and Testing	44
5.1	User Interface and Snapshots	45
5.2	Test Cases and Result	50
6	Conclusion and Future Scope	54
-	References	57

List Of Figures

Figure No.	Title	Page No.
3.1	Use Case Diagram of Buildr	20
3.2	Context-Level DFD of Buildr	21
3.3	Level 1 DFD of Buildr	23
3.4	Class Diagram of Buildr	25
3.5	Entity Relationship Diagram of Buildr	27
3.6	Activity Diagram of Buildr	29
5.1	Home Page of Buildr	45
5.2	Project Workspace of Buildr	46
5.3	Generated Website Preview	47
5.4	Generated Code Explorer	48

List of Tables

Table No.	Title	Page No.
1.1	Hardware Requirements	7
1.2	Software Requirements	8
2.1	Feasibility Study	13
2.2	Project Timeline	14
3.1	Data Dictionary for Project Table	31
3.2	Data Dictionary for Message Table	31
3.3	Data Dictionary for Fragment Table	32
5.1	Functional Test Cases	51

Abbreviations and Nomenclature

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CSS	Cascading Style Sheets
DFD	Data Flow Diagram
ER	Entity Relationship
HMR	Hot Module Replacement
JSON	JavaScript Object Notation
ORM	Object Relational Mapping
PDF	Portable Document Format
SRS	Software Requirement Specification
trPC	TypeScript Remote Procedure Call
UI	User Interface

Abstract of the Project

Buildr is an AI-powered web application builder that converts natural language prompts into complete website prototypes. The system allows a user to describe a website requirement, creates a project automatically, sends the work to an AI coding workflow, runs generated code inside an isolated sandbox and displays both a live preview and generated source code in the browser.

The project has been developed using Next.js, React, TypeScript, Tailwind CSS, Shadcn/UI, tRPC, Prisma, PostgreSQL, Clerk, Inngest, OpenAI and E2B. These tools together provide a modern full-stack architecture with authentication, project management, type-safe APIs, database persistence, background processing, usage tracking and safe code execution.

This report follows the prescribed semester training format. It contains the introduction, system analysis, system design, software tools, implementation details, testing, conclusion, future scope and references. The report also includes hardware and software requirements, feasibility study, diagrams, data dictionary and test cases related to the Buildr project.

Buildr demonstrates how artificial intelligence can support rapid software prototyping. It is useful for students, developers, entrepreneurs and non-technical users who need to convert ideas into usable web interfaces quickly while still being able to inspect the generated code.

Chapter 1 : Introduction

Buildr is a full-stack AI website builder designed to reduce the effort required to create a working web prototype. A user enters a prompt such as a portfolio website, business landing page or dashboard, and the system converts that requirement into generated website files and a running preview.

The project is based on the idea that modern artificial intelligence can act as a coding assistant when it is connected to a real development environment. Buildr provides that environment through a Next.js application, a database, a background workflow and an E2B sandbox.

Unlike a simple chatbot, Buildr stores projects and messages, generates code in an isolated workspace and displays the result through a preview and file explorer.

1.1 Brief Overview of Work

The work completed in this project includes design and implementation of an AI-powered project generation system. The application begins with a prompt form on the home page. After authentication, the user can create projects, submit messages, view generated results and continue improving the website through follow-up prompts.

The backend records every project and message using Prisma and PostgreSQL. The generation task is sent to Inngest because website generation may take more time than a normal web request. The Inngest function creates a sandbox, runs an AI coding agent, saves generated files and stores the final result in the database.

The user interface is divided into a message area, a live demo area and a code viewing area. This layout helps the user understand both the final result and the source files produced by the AI agent.

1.2 Objective

The main objective of Buildr is to provide a web platform that can generate a functional website from a natural language prompt. The project also aims to make AI generation practical by connecting it with authentication, database storage, background processing and live preview.

The system is intended to support quick prototyping and learning. Users can experiment with ideas, observe the generated output and inspect the code structure without setting up a project manually.

- To accept website requirements in natural language.
- To create and maintain user-specific projects.
- To run AI code generation in the background.
- To execute generated code in an isolated sandbox.
- To display live preview and generated source code.
- To maintain usage control for fair resource consumption.

1.3 Scope

The scope of the project includes user authentication, prompt submission, project creation, message history, background AI generation, sandbox execution, result storage and preview display. Buildr is suitable for generating early-stage web prototypes and small application interfaces.

The project does not claim to replace complete software engineering practices. Generated code should be reviewed before production use. However, it significantly reduces the time needed to move from an idea to a demonstrable web interface.

1.4 Project Modules

The Buildr project is divided into multiple modules so that each responsibility is clear. The home module manages project creation, the project module handles project retrieval, the message module stores conversations, the Inngeist module performs AI generation and the UI components render preview and code.

1.4.1 Authentication

Authentication is managed through Clerk. Users must sign in before creating or opening protected projects. The authenticated user identifier is stored with each project and checked in protected server procedures.

1.4.2 Prompt Processing

Prompt processing starts when the user submits a text requirement. The input is validated, usage credits are consumed and a project or message record is created. The same prompt is then sent to the background generation function through an Inngeist event.

1.4.3 Project and Message Management

Project management stores project names, owners and timestamps. Message management stores both user prompts and assistant responses. These records allow the user to return to previous work and continue improving a generated website.

1.4.4 Preview and Code Explorer

The preview module displays the generated website through an iframe using the sandbox URL. The code explorer reads stored fragment files and presents them in a file tree with syntax-highlighted content.

1.5 Project Requirements

The project requirements include hardware and software resources needed for development, execution and usage. Since Buildr is a web application, client-side requirements are modest, while the backend requires API keys and service configuration.

Table 1.1: Hardware Requirements

Component	Minimum Requirement	Recommended Requirement
Processor	Dual-core processor	Quad-core processor or higher
RAM	4 GB	8 GB or higher
Storage	1 GB free space	5 GB free space
Network	Internet connection	Stable broadband connection
Client	Modern web browser	Chrome, Edge or Firefox

1.5.1 Software

The software requirements include the tools and services needed to build and run the application. Node.js and npm are used for development, while the application framework and backend services handle user-facing and server-side behaviour.

Table 1.2: Software Requirements

Software	Purpose
Next.js and React	Application framework and interface
TypeScript	Type-safe development
Tailwind CSS and Shadcn/UI	Styling and reusable components
tRPC and React Query	Type-safe API communication
Prisma and PostgreSQL	Database schema and storage
Clerk	Authentication
Inngest	Background workflow
OpenAI	AI generation
E2B	Sandbox execution

Chapter 2 : System Analysis

System analysis studies the existing problem, evaluates feasibility and defines how Buildr should behave. The project solves the problem of slow manual prototyping by using AI agents and sandbox execution.

2.1 Literature Review

Traditional website development requires knowledge of HTML, CSS, JavaScript, frameworks, component libraries, state management, database access and deployment. Even a simple prototype may take significant time if created manually.

Recent developments in large language models have shown that AI can generate code from natural language. However, raw code generation is not enough. A practical system must validate input, run the generated application, store the output and show it to the user. Buildr combines these requirements into one integrated platform.

Modern full-stack frameworks such as Next.js help developers build both frontend and backend parts in one codebase. Tools such as tRPC, Prisma and Inngest make the architecture more reliable by providing typed APIs, database access and background execution.

Importance of AI Website Builders

AI website builders are important because they reduce the distance between idea and implementation. Students can use them to learn project structure, designers can test layouts quickly and entrepreneurs can validate product ideas before investing in complete development.

Buildr is also useful as an educational tool because it exposes generated files instead of hiding the code. Users can compare their prompt with the generated output and understand how components are organised.

2.2 Project Feasibility Study

The feasibility study determines whether Buildr is practical from technical, economic and operational perspectives. The selected technologies are widely used and can be integrated through APIs and SDKs.

Table 2.1: Feasibility Study

Type	Description
Technical Feasibility	Achievable with Next.js, Prisma, Inngest, OpenAI and E2B
Economic Feasibility	Uses open-source tools with paid external services only where required
Operational Feasibility	Prompt-based workflow is simple for end users
Schedule Feasibility	Modular design allows staged development and testing

2.2.1 Technical Feasibility

The project is technically feasible because the required functionality can be implemented using existing libraries and cloud services. Next.js supports full-stack routes, Clerk supports authentication, Prisma supports database access, Inngest supports background jobs and E2B supports isolated execution.

2.2.2 Economical Feasibility

The project is economically feasible because most development tools are open-source. Costs are mainly associated with AI model usage, sandbox execution and hosting. Usage tracking can control resource consumption.

2.2.3 Operational Feasibility

The system is operationally feasible because users interact with a simple chat-like workflow. They do not need to understand the internal AI or sandbox process. They only need to enter requirements and review the generated result.

2.4 Project Timeline Chart

The project was completed in phases including requirement study, design, implementation, testing and report preparation.

Table 2.2: Project Timeline

Phase	Work Completed
Week 1	Requirement collection and project study
Week 2	Technology selection and database design
Week 3	Authentication and project module
Week 4	Message flow and tRPC procedures
Week 5	Inngest agent and sandbox integration
Week 6	Preview, code explorer and testing
Week 7	Documentation and final report

2.5 Detailed Module Description with all Functionalities

The home module provides the first interaction point. It accepts a prompt and shows a list of existing projects. The project module verifies ownership and loads project details. The message module stores user and assistant communication.

The background generation module is responsible for sandbox creation, previous message loading, AI agent execution, title generation, response generation and result saving. The usage module prevents unlimited generation by consuming credits.

2.5.1 Registration and Login

Registration and login are provided by Clerk. After successful login, the user receives an authenticated session. The application uses the session to allow access to protected routes and server procedures.

2.5.2 Project Creation

Project creation happens when a user submits the first prompt. The system validates the prompt, creates a project name, stores the message and starts the generation workflow.

2.5.3 Search and Project Listing

The project listing feature displays projects belonging to the logged-in user. Projects are ordered by update time so that recent work appears first.

2.5.4 Manage Account and Usage

User account management is handled by Clerk. Usage management is handled by the credit system, which limits the number of generations available to a user.

Chapter 3 : System Design

System design describes the structural view of Buildr. It includes use case design, data flow diagrams, class relationships, entity relationship diagram, activity diagram and data dictionary.

3.1 Use Case Diagrams

The use case diagram identifies the main actions performed by the user. The user can sign in, create a project, submit prompts, view messages, preview generated websites and inspect code.

Actor: User

Use Cases: Login, Create Project, Submit Prompt, View Preview, Inspect Code, Continue Iteration

External Services: Clerk, Inngest, OpenAI, E2B

Figure 3.1: Use Case Diagram of Buildr

3.2 Data Flow Diagrams

Data flow diagrams explain how information moves through the system. The prompt flows from the UI to the tRPC API, then to the database and background generation function.

User -> Buildr Web UI -> tRPC API
tRPC API -> PostgreSQL Database
tRPC API -> Inngest Event
Inngest -> OpenAI Agent + E2B Sandbox
Sandbox Result -> Database Fragment -> User Preview

Figure 3.2: Context-Level DFD of Buildr

3.2.1 Context-Level (Level 0)

At context level, Buildr is represented as a single system interacting with the user and external services. The user provides prompts and receives generated websites. External services provide identity, AI generation and sandbox execution.

3.3.2 Level 1 DFD

At level 1, the system is divided into authentication, project management, message management, background generation and preview display. Each process communicates with the database or an external service as required.

1. Authentication validates user
2. Project module stores project
3. Message module stores prompt
4. Inngest runs AI generation
5. Fragment module stores URL and files
6. UI renders preview and code

Figure 3.3: Level 1 DFD of Buildr

3.3 Class Diagram

The class diagram represents the logical structure of the system. Project, Message, Fragment and Usage are the most important data classes. UI components and server procedures operate on these classes through Prisma.

Project: id, name, userId, createdAt, updatedAt

Message: id, content, role, type, projectId

Fragment: id, messageId, sandboxUrl, title, files

Usage: key, points, expire

Figure 3.4: Class Diagram of Buildr

3.4 Entity Relationship Diagram

The ER diagram shows that one project can have many messages and one message can have one fragment. Usage is a separate table used for credit tracking.

Project 1 ---- Many Message
Message 1 ---- Zero or One Fragment
Usage stores credit information independently

Figure 3.5: Entity Relationship Diagram of Buildr

3.5 Activity Diagram

The activity diagram describes the flow from user login to generated website preview. It also includes the decision point where generation may succeed or fail.

Start -> Login -> Enter Prompt -> Create Project -> Send Event
Create Sandbox -> Run AI Agent -> Files Generated?
Yes -> Save Fragment -> Display Preview
No -> Save Error Message -> Display Error

Figure 3.6: Activity Diagram of Buildr

3.6 Data Dictionary

The data dictionary describes the main database fields used by Buildr. These tables are defined in the Prisma schema.

Table 3.1: Data Dictionary for Project Table

Field	Type	Description
id	String	Unique project identifier
name	String	Generated project name
userId	String	Owner user identifier
createdAt	DateTime	Creation timestamp
updatedAt	DateTime	Last update timestamp

3.6.1 Message and Fragment Tables

Messages store conversation data and fragments store generated output. These two tables connect the user-facing chat with the generated website result.

Table 3.2: Data Dictionary for Message Table

Field	Type	Description
id	String	Unique message identifier
content	String	Prompt or assistant response
role	Enum	USER or ASSISTANT
type	Enum	RESULT or ERROR
projectId	String	Related project identifier

3.6.2 Fragment Table

A fragment is created when AI generation succeeds. It contains the sandbox URL, generated title and the file map used by the code explorer.

Table 3.3: Data Dictionary for Fragment Table

Field	Type	Description
id	String	Unique fragment identifier
messageId	String	Related assistant message
sandboxUrl	String	Live preview URL
title	String	Generated fragment title
files	Json	Generated source files

Chapter 4 : Software Tools

This chapter explains the software tools and technologies used in Buildr. The project uses a modern TypeScript-based full-stack environment with AI and sandbox services.

4.1 Overview of Next.js Technology

Next.js is the main framework used in Buildr. It provides routing, layouts, API routes and server-side capabilities in one application. The App Router structure keeps pages and API endpoints organised.

4.1.1 Features of Next.js

Important features include file-based routing, server components, client components, hot reload, API routes and production optimisation. These features make Next.js suitable for a full-stack AI builder platform.

4.1.2 React and Component Based UI

React is used to build reusable interface components such as forms, message cards, tabs, panels and file explorers. Its component model keeps the user interface maintainable.

4.1.3 TypeScript

TypeScript adds static typing to JavaScript. It improves reliability by checking component props, API inputs, database types and utility function signatures during development.

4.2 About Tailwind CSS

Tailwind CSS is a utility-first styling framework. Buildr uses it to create responsive layouts, spacing, colours and typography quickly while maintaining a consistent design.

4.3 Shadcn/UI and Radix UI

Shadcn/UI provides reusable interface components based on Radix UI primitives. These components support accessibility and allow direct customisation because the component code exists inside the project.

4.4 tRPC

tRPC provides type-safe communication between frontend and backend. Procedures are defined on the server and called from the client without manually writing separate REST schemas.

4.5 Prisma and PostgreSQL

Prisma is the object relational mapper used to access the PostgreSQL database. The Prisma schema defines Project, Message, Fragment and Usage models.

4.6 Clerk Authentication

Clerk manages user sign-in, sign-up and session handling. Buildr uses Clerk user identifiers to protect projects and API procedures.

4.7 Inngest

Inngest is used for background job execution. The AI generation process runs as an event-driven function so that long tasks do not block the main request.

4.8 OpenAI and Agent Kit

OpenAI models are used for code generation, response generation and title generation. Agent Kit provides a tool-based agent workflow that can read files, write files and run commands.

4.9 E2B Sandbox

E2B provides an isolated code execution environment. Generated applications run inside the sandbox and expose a preview URL for the Buildr interface.

4.10 Development Tools

The project also uses npm, ESLint, TypeScript configuration, Prisma migrations, Docker files and sandbox templates. These tools help with development, quality control and deployment preparation.

Chapter 5 : Implementation and Testing

Implementation converts the system design into working software. Buildr has been implemented through frontend modules, server procedures, database schema, background functions and sandbox templates.

Implementation

The implementation starts from the home page where the user enters a prompt. The prompt is sent to the projects router, which validates the input, consumes credit, creates a project and sends a BuildrAgent/run event.

The background function creates a sandbox, loads previous messages, runs the AI coding agent and stores the generated result. The project page then shows messages, preview and code.

5.1 User Interface and Snapshots

The user interface is designed around the builder workflow. The first screen accepts a prompt, while the project screen contains the chat history, preview and source code.

Home page contains navigation, user control, project form and project list.

The prompt box accepts natural language requirements for website generation.

Figure 5.1: Home Page of Buildr

5.1.1 Project Workspace

The project workspace displays the conversation on the left and the generated output on the right. The right panel contains tabs for demo preview and code view.

Messages Panel | Demo and Code Panel
Prompt Input | Generated Website Preview | File Explorer

Figure 5.2: Project Workspace of Buildr

5.1.2 Generated Website Preview

The generated website preview is loaded from the sandbox URL. This gives immediate visual feedback after generation completes.

Sandbox URL -> iframe preview

User can view the generated application directly in the Buildr project page.

Figure 5.3: Generated Website Preview

5.1.3 Generated Code Explorer

The code explorer displays generated files stored in the fragment. This feature makes the system transparent and helps users learn from the generated source code.

File Tree -> Selected File -> Syntax Highlighted Code
Generated files are stored as JSON in the Fragment record.

Figure 5.4: Generated Code Explorer

5.2 Test Cases and Result

Testing was performed to verify that the main workflows behave correctly. The test cases cover authentication, project creation, message submission, background generation and preview display.

Table 5.1: Functional Test Cases

Test Case	Expected Result	Status
Valid login	User reaches protected area	Pass
Submit valid prompt	Project and message are created	Pass
Submit empty prompt	Validation error is shown	Pass
Open own project	Project details are loaded	Pass
Open another user's project	Access is denied	Pass
Generation success	Fragment is saved and preview is shown	Pass
Generation failure	Error message is stored	Pass

Unit Testing

Unit testing focuses on individual functions and components. Input validation, utility functions, UI components and API procedure behaviour can be tested independently.

Integration Testing

Integration testing verifies that modules work together. For Buildr, important integration points include tRPC with Prisma, Clerk with protected procedures, Inngest with the database and the sandbox with generated output.

Functional Testing

Functional testing verifies user-visible requirements. The application was checked for project creation, message display, preview rendering and code inspection.

Acceptance Testing

Acceptance testing verifies whether the system satisfies the project objectives. Buildr accepts prompts, generates output and stores results, so it meets the main acceptance criteria.

Conversion

Conversion refers to moving from manual website prototyping to an AI-assisted workflow. Buildr supports this by automating setup, generation and preview while still allowing human review.

Post Implementation Review

After implementation, the system was reviewed for usability, maintainability and reliability. The modular structure makes future improvements easier, and the background workflow improves reliability for long-running generation tasks.

Chapter 6 : Conclusion and Future Scope

This chapter summarises the project outcome and presents the future scope of Buildr.

Conclusion

Buildr successfully implements an AI-powered website builder that converts prompts into live web previews. It combines a modern frontend, protected server procedures, database persistence, background AI execution and sandboxed code running.

The project fulfils its objective of reducing the effort required for web prototyping. It also provides educational value because generated code can be inspected directly from the browser.

Benefits:

Buildr offers benefits to both technical and non-technical users. It saves time, supports experimentation and gives quick visual output from written requirements.

- Fast conversion of ideas into website prototypes.
- Project history and iterative prompting.
- Safe sandbox execution of generated code.
- Transparent code viewer for learning and review.
- Modern and maintainable full-stack architecture.

Future Work

Future improvements can include GitHub export, direct deployment, version history, team collaboration, better generated-code testing, custom design controls, file download, reusable templates and stronger monitoring of AI output quality.

The project can also be extended with multi-page application generation, database-backed generated apps and automatic documentation for each generated website.

References

The following references were used for project understanding and report preparation.

Books

1. Software Engineering by Roger S. Pressman.
2. Database System Concepts by Abraham Silberschatz, Henry F. Korth and S. Sudarshan.
3. Web Engineering: A Practitioner's Approach by Roger S. Pressman and David Lowe.

Articles/Journals

1. Articles related to AI-assisted software development and prompt-based code generation.
2. Technical articles on full-stack TypeScript application architecture.
3. Documentation notes prepared from the Buildr project source code.

Websites

1. <https://nextjs.org/>
2. <https://react.dev/>
3. <https://www.prisma.io/>
4. <https://clerk.com/>
5. <https://www.inngest.com/>
6. <https://openai.com/>
7. <https://e2b.dev/>